

Machine Learning

Winter 2011/12

Roland Memisevic

Lecture 11, Jan. 23, 2012

Non-parametric methods

- ▶ All learning approaches we looked at so far were based on parameterizing a model using parameters, θ , and using the training data to adapt θ .
- ▶ **Non-parametric** methods are a large class of methods that *use the training data-set itself* to define a model.
- ▶ (They may or may not contain some parameters in addition).
- ▶ There are non-parametric methods for all common learning tasks like regression, classification and unsupervised learning.
- ▶ Main Pro: They make it easy to define highly non-linear dependencies in data. Main Con: The computational complexity of making predictions grows with the training set size, and the complexity of learning often even quadratically or worse.

Outline

- ▶ Nearest neighbors
- ▶ Kernel densities
- ▶ Duality
- ▶ Support vector machines
- ▶ Loss functions

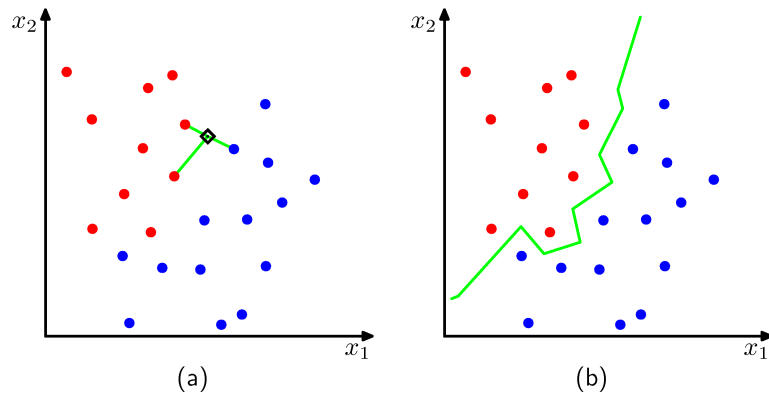
K-Nearest Neighbor Classification

- ▶ Given training features $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and corresponding class labels $\{t_1, \dots, t_N\}$, K-nearest neighbors defines the classification decision for a test-point $\mathbf{x}$ as follows:

K-nearest neighbors (KNN)

- ▶ Compute the distance $d(\mathbf{x}, \mathbf{x}_i)$ between $\mathbf{x}$ and all training cases. A common choice is Euclidean distance:
$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|$$
- ▶ Predict as the label for $\mathbf{x}$ the majority of labels among the K nearest training cases.
- ▶ One way to deal with ties: back off to $K - 1$. Another way: random choice among the winning labels.

KNN Example



- ▶ (a) $K = 3$. The test-point gets the same label as the red points.
- ▶ (b) Classification boundary resulting from a 1-NN classifier.

KNN comments

- ▶ There is no learning: Just save off the training set.
- ▶ The computational complexity for making predictions is linear in the training set size.
- ▶ A simple way to speed up the predictions: For each prediction, keep track of the K^{th} smallest distance so far; back out of the distance computation for case $\mathbf{x}_i$, if the partial computation of $\|\mathbf{x} - \mathbf{x}_i\|$ is already larger than this.
- ▶ Another common speed-up technique: *KD-trees*.
- ▶ Although NN methods are most commonly used in classification, we can perform nearest-neighbor regression, too: Assign to test case $\mathbf{x}$ the (average) real value t of the K nearest neighbor(s) of $\mathbf{x}$. This leads to a piece-wise constant model.

Kernel density estimation

- ▶ One way to “smooth out” the predictions of nearest neighbor methods is to use

Kernel density estimation (KDE)

- ▶ A kernel density estimate models the density of data as a normalized superposition of local “bumps” centered at the training points:

$$p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n)$$

- ▶ The function $k(\mathbf{x}, \mathbf{x}_n)$ is known as a “kernel function”, or a “Parzen window”, and it needs to satisfy $\int k(\mathbf{x}, \mathbf{x}_n) d\mathbf{x} = 1$ and $k(\mathbf{x}, \mathbf{x}_n) \geq 0$

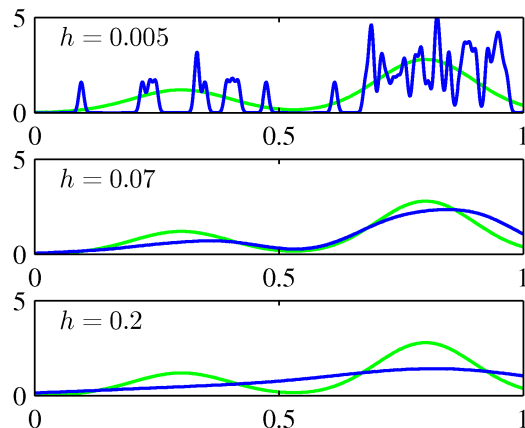
Kernel density estimation

- ▶ A common choice of kernel function is the Gaussian kernel:

$$k(\mathbf{x}, \mathbf{x}_n) = \frac{1}{(2\pi h^2)^{D/2}} \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2h^2}\right)$$

- ▶ This is a Gaussian centered at data-point $\mathbf{x}_n$.
- ▶ To set kernel hyper-parameters, such as the variance, we can use cross-validation.
- ▶ A KDE with Gaussian kernels may be thought of as a Gaussian mixture model with N mixture components.
- ▶ A KDE is like a “smoothed histogram”:

Gaussian KDE example



The Nadaraya-Watson regression model

- ▶ Given a joint KDE over inputs $\mathbf{x}$ and outputs t , we can derive a regression model by simply computing the conditional over t .
- ▶ It is convenient for this purpose to define the kernel over $\mathbf{x}$ and t as the product $k(\mathbf{x}, \mathbf{x}_n)k(t, t_n)$. Note that this will automatically be the case for the (spherical) Gaussian kernel we defined before.
- ▶ The conditional over outputs is then given by

$$p(t|\mathbf{x}) = \frac{p(\mathbf{x}, t)}{\int p(\mathbf{x}, t) dt} = \frac{\sum_n k(\mathbf{x}, \mathbf{x}_n)k(t, t_n)}{\sum_m k(\mathbf{x}, \mathbf{x}_m)}$$

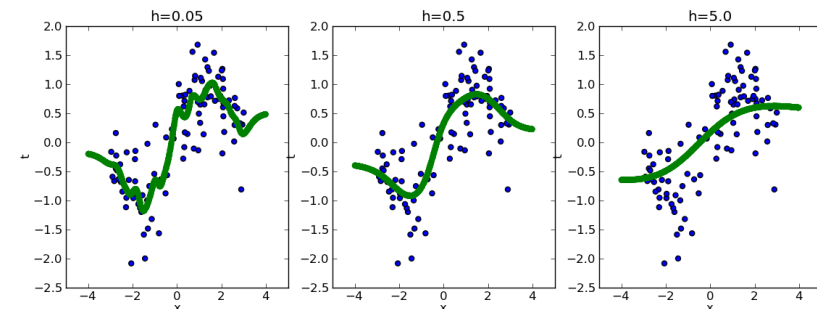
The Nadaraya-Watson regression model

- ▶ When making predictions, we typically care the most about the conditional expectation.
- ▶ For symmetric kernels (such as the Gaussian), this is given by:

$$y(\mathbf{x}) := \mathbb{E}[t|\mathbf{x}] = \sum_n \frac{k(\mathbf{x}, \mathbf{x}_n)}{\sum_m k(\mathbf{x}, \mathbf{x}_m)} t_n$$

- ▶ This is known as the *Nadaraya-Watson* regression model.
- ▶ Make predictions by adding up all training outputs, weighted by how close they are to the query point $\mathbf{x}$.
- ▶ It is also known as “kernel smoothing”.
- ▶ Since the normalizing constants cancel, the kernel functions no longer need to be normalized.
- ▶ One can also get the conditional variance in closed form.

Nadaraya-Watson regression example



KDE comments

- ▶ As with nearest-neighbor methods, predictions typically scale with $O(N)$, and there is no learning phase, except for setting hyper-parameters.
- ▶ To derive a (generative) classifier based on a KDE, just use the KDE as class-conditionals.
- ▶ KDEs are based on “density kernels”, which define a local “part” of a density.
- ▶ There is another kind of kernel function, sometimes referred to as “Mercer kernels”, which are not related to density estimation. Their usefulness comes from in the fact that they implicitly compute an inner product in a (possibly high-dimensional) feature space.
- ▶ There is very a large class of methods that make use of this fact.

Dual representation of linear regression

- ▶ By substituting $\mathbf{w} = \mathbf{X}^T \mathbf{a}$ in the objective we get

$$E(\mathbf{a}) = \frac{1}{2} \mathbf{a}^T \mathbf{X} \mathbf{X}^T \mathbf{X} \mathbf{X}^T \mathbf{a} - \mathbf{a}^T \mathbf{X} \mathbf{X}^T \mathbf{t} - \frac{1}{2} \mathbf{t}^T \mathbf{t} + \frac{\lambda}{2} \mathbf{a}^T \mathbf{X} \mathbf{X}^T \mathbf{a}$$

with $\mathbf{t} = (t_1, \dots, t_N)^T$

- ▶ The objective suggests that we may learn $\mathbf{a}$ instead of $\mathbf{w}$!
- ▶ The data-points enter in the objective only as inner products, so by defining the matrix of inner products $\mathbf{K} = \mathbf{X} \mathbf{X}^T$ we can simplify it to

$$E(\mathbf{a}) = \frac{1}{2} \mathbf{a}^T \mathbf{K} \mathbf{K} \mathbf{a} - \mathbf{a}^T \mathbf{K} \mathbf{t} + \frac{1}{2} \mathbf{t}^T \mathbf{t} + \frac{\lambda}{2} \mathbf{a}^T \mathbf{K} \mathbf{a}$$

- ▶ Setting the derivative to zero shows that the optimum is given by

$$\mathbf{a} = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t}$$

Dual representation of linear regression

- ▶ Consider the regularized linear regression cost

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}_n - t_n)^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

- ▶ Setting the gradient to zero allows us to write

$$\mathbf{w} = -\frac{1}{\lambda} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}_n - t_n) \mathbf{x}_n =: \sum_{n=1}^N a_n \mathbf{x}_n = \mathbf{X}^T \mathbf{a}$$

where

$$a_n = -\frac{1}{\lambda} (\mathbf{w}^T \mathbf{x}_n - t_n)$$

- ▶ This shows that the optimal parameter vector will be given by a *linear combination of the training points*.

Making predictions with the dual representation

- ▶ After learning $\mathbf{a}$, we can make predictions using

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \mathbf{a}^T \mathbf{X} \mathbf{x}$$

- ▶ This also involves inner products: between the test-case $\mathbf{x}$ and all training cases.
- ▶ If we stack the inner products in the vector $\mathbf{k}(\mathbf{x})$ we can write

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^T (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t}$$

- ▶ This seems like a strange way to re-formulate the linear regression task.
- ▶ But dealing only with inner products can be useful, because it allows us to utilize the “kernel trick”:

The “kernel trick”

Kernels

- ▶ We call a symmetric function $k(\cdot, \cdot)$ a positive definite kernel function, if for *any* set of points $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, the matrix $\mathbf{K}$ with entries $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is positive definite.
- ▶ These kernel functions have a different purpose than those we used for the KDE, and they do not need to be normalized. Their usefulness is due to the:

“Kernel Trick”

- ▶ A positive definite kernel can be shown to correspond to the *inner product* in a feature-space related to the input space (eg., “Mercer’s Theorem”).
- ▶ In other words: $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}')$ for some, possibly non-linear, function $\phi(\cdot)$.

Constructing kernels

- ▶ There are many rules which are known to preserve positive definiteness and which allow us to construct new kernels $k(\cdot, \cdot)$ from known kernels $k_1(\cdot, \cdot), k_2(\cdot, \cdot)$:
 1. $k(\mathbf{x}, \mathbf{x}') = ck_1(\mathbf{x}, \mathbf{x}')$
 2. $k(\mathbf{x}, \mathbf{x}') = \exp(k_1(\mathbf{x}, \mathbf{x}'))$
 3. $k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$
 4. etc.
- ▶ See (Bishop, page 296) for more.
- ▶ A very common choice of kernel function is the Radial Basis Function (RBF)- or ‘Gaussian’ kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2 / 2\sigma^2)$$

which is an unnormalized version of the Gaussian density kernel we have seen before.

- ▶ The feature space of the Gaussian kernel can be shown to be infinite dimensional.

Kernel trick example

- ▶ This means that kernels allow us to deploy non-linear features without actually computing them!
- ▶ As an example, consider the following kernel function

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}')^2$$

- ▶ In 2 – D, we have:

$$\begin{aligned} k(\mathbf{x}, \mathbf{x}') &= (x_1 x'_1 + x_2 x'_2)^2 \\ &= x_1^2 x_1'^2 + 2x_1 x'_1 x_2 x'_2 + x_2^2 x_2'^2 \\ &= (x_1^2, \sqrt{2}x_1 x_2, x_2^2) (x_1'^2, \sqrt{2}x'_1 x'_2, x_2'^2)^T := \phi(\mathbf{x})^T \phi(\mathbf{x}') \end{aligned}$$

- ▶ Thus, this kernel allows us to use a quadratic basis expansion (with some weighting) without actually computing the quadratic features.
- ▶ In practice, we may use a kernel function and not know (or even care) what the feature space looks like.

Writing models in terms of inner products

- ▶ Whenever we can re-formulate a method so that data enters only in terms of inner products, we can make use of the kernel trick.
- ▶ We have seen this for regularized linear regression.
- ▶ Another Example: KNN is based on the distances $\|\mathbf{x}_1 - \mathbf{x}_2\|^2$ which we can compute as $\mathbf{x}_1^T \mathbf{x}_1 + \mathbf{x}_2^T \mathbf{x}_2 - 2\mathbf{x}_1^T \mathbf{x}_2$
- ▶ Another example: PCA computes “eigen-features” $\mathbf{v}$ with $\mathbf{X}^T \mathbf{X} \mathbf{v} = \lambda \mathbf{v}$. Multiply from the left with $\mathbf{X}$ to get: $\mathbf{X} \mathbf{X}^T \mathbf{X} \mathbf{v} = \lambda \mathbf{X} \mathbf{v}$. So, if diagonalize $\mathbf{X} \mathbf{X}^T$ instead, we will directly get the latent projections $\mathbf{z} = \mathbf{X} \mathbf{v}$. This is known as **kernel PCA**.
- ▶ One can “kernelize” many other methods, including logistic regression, K-means clustering, etc.

Support vector machines

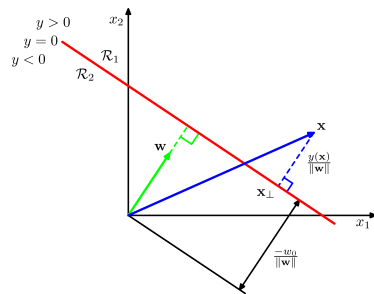
- ▶ **Support vector machines** (SVMs) played a central role in popularizing kernel methods, and so they are commonly viewed as prototypical kernel methods.
- ▶ But in their basic, “non-kernelized” form, SVMs are just another way to train a linear classifier.
- ▶ Recall that we can define a *two-class* classifier using the linear function

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

and a decision rule that looks at the sign of $y(\mathbf{x})$.

- ▶ Support vector machines try to set the parameters such that the **margin**, defined as the perpendicular distance between the decision boundary and the closest data-point, becomes maximal:

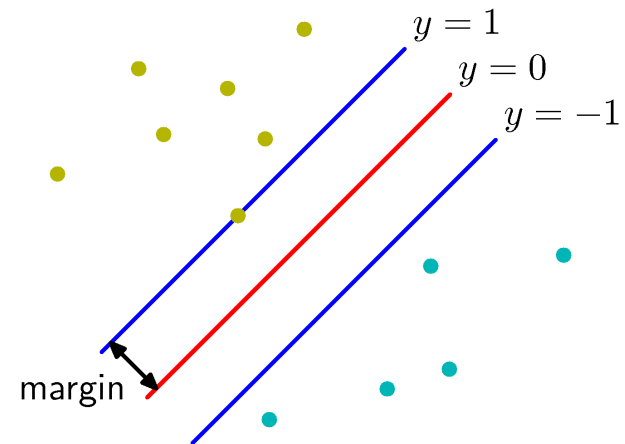
Maximizing the margin



- ▶ Since points on the separating hyperplane satisfy $\mathbf{w}^T \mathbf{x} + b = 0$, the distance of the hyperplane from the origin is $\frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\|} = -\frac{b}{\|\mathbf{w}\|}$
- ▶ So the distance of any point $\mathbf{x}$ from the hyperplane is

$$|y(\mathbf{x})| / \|\mathbf{w}\|$$

The margin



Maximizing the margin, separable case

- ▶ If data is linearly separable, then the optimal solution will also satisfy $t_n y(\mathbf{x}_n) > 0$ for all n .
- ▶ So we can write the distance of training points from the hyperplane as

$$\frac{t_n (\mathbf{w}^T \mathbf{x}_n + b)}{\|\mathbf{w}\|}$$

- ▶ This allows us to define the maximum margin parameters as the solution to the following optimization problem:

$$\arg \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|} \min_n [t_n (\mathbf{w}^T \mathbf{x}_n + b)]$$

- ▶ The min makes this a hard problem to solve!

SVM primal optimization problem

- ▶ To simplify it, note that the distance to the hyperplane won't change if we replace $\mathbf{w} \rightarrow \kappa \mathbf{w}$ and $b \rightarrow \kappa b$.
- ▶ So we are free to fix the scale at any value.
- ▶ Let us assume the scale is such that the closest point satisfies

$$t_n(\mathbf{w}^T \mathbf{x}_n + b) = 1$$

- ▶ In this case *all* points will satisfy $t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1$.
- ▶ This allows us to write the problem as a *constrained optimization problem*:

Primal SVM optimization problem (separable case)

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & t_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1 \end{aligned}$$

Deriving the dual problem

- ▶ To train the SVM we could solve this problem directly.
- ▶ But it is more common to make use of the convexity of this problem and to solve the *dual problem*, which can be derived by writing down the Lagrangian:

$$L(\mathbf{w}, b, \mathbf{a}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=1}^N a_n (t_n(\mathbf{w}^T \mathbf{x}_n + b) - 1)$$

where a_n are Lagrange multipliers.

- ▶ Setting derivatives to zero yields

$$\mathbf{w} = \sum_{n=1}^N a_n t_n \mathbf{x}_n \quad \text{and} \quad 0 = \sum_{n=1}^N a_n t_n$$

which makes it possible to eliminate $\mathbf{w}$ and b to arrive at the dual:

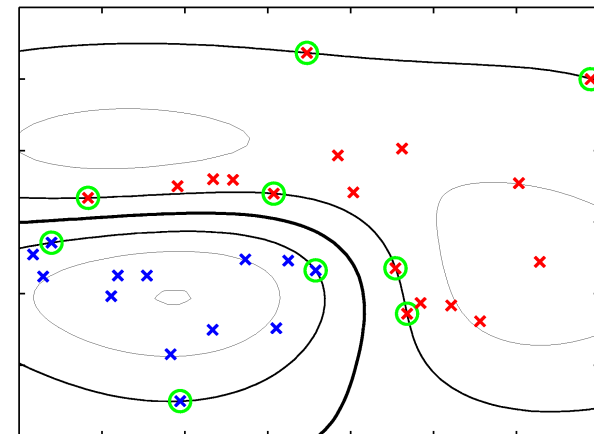
SVM dual problem

Dual SVM optimization problem (separable case)

$$\begin{aligned} \text{maximize} \quad & \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m t_n t_m \mathbf{x}_n^T \mathbf{x}_m \\ \text{s.t.} \quad & a_n \geq 0 \\ & \sum_{n=1}^N a_n t_n = 0 \end{aligned}$$

- ▶ This problem is known as a *Quadratic Program*: A quadratic objective function with linear constraints.
- ▶ Standard methods exist for solving it efficiently.
- ▶ In this problem, the data appears only in inner products, and so we can use kernels.

Classification boundary resulting from RBF kernel



Applying the SVM to test data

- To apply the model to test data, we can back-substitute a in the equation for $\mathbf{w}$ to get:

$$y(\mathbf{x}) = \sum_{n=1}^N a_n t_n k(\mathbf{x}, \mathbf{x}_n) + b$$

- Like with other non-parametric models, this grows linearly in the number of training points.
- In practice, however, many of the a_n tend to be exactly 0 at the optimum (hence the name “support vector machine”).
- Several approaches to extending the SVM to deal with multi-class problems have been suggested, such as training K SVMs which separate each class from all other classes.
- What if the training set is not linearly separable?

SVM non-separable dual

Dual SVM optimization problem (non-separable case)

$$\begin{aligned} \text{maximize} \quad & \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m t_n t_m \mathbf{x}_n^T \mathbf{x}_m \\ \text{s.t.} \quad & 0 \leq a_n \leq C \\ & \sum_{n=1}^N a_n t_n = 0 \end{aligned}$$

- The classification rule is the same as before.

SVM non-separable case

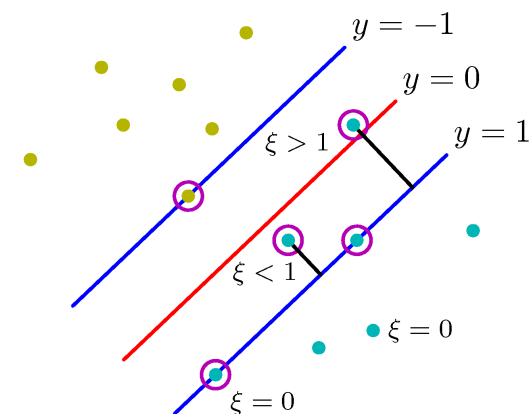
- We can use *slack variables* ξ_n to turn the constraints into “soft-constraints”:

Primal SVM optimization problem (non-separable case)

$$\begin{aligned} \text{minimize} \quad & C \sum_{n=1}^N \xi_n + \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & t_n (\mathbf{w}^T \mathbf{x}_n + b) \geq 1 - \xi_n \end{aligned}$$

- We need to set C . Standard approach: cross-validation.
- Again, we can compute the Lagrangian and get at a quadratic program as the dual:

Slack variable intuition



SVM training by unconstrained optimization

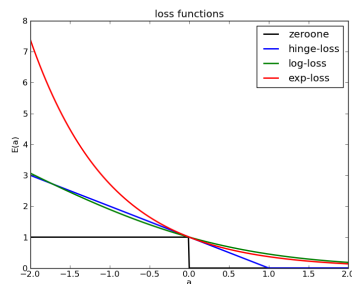
- ▶ For points on the correct side of the margin boundary we have $y(\mathbf{x}_n)t_n \geq 1$, so $\xi_n = 0$, and for points on the wrong side we have $\xi_n = 1 - y(\mathbf{x}_n)t_n$.
- ▶ We can therefore reformulate the SVM optimization problem as the *unconstrained problem* of minimizing the cost:

$$\sum_{n=1}^N [1 - y(\mathbf{x}_n)t_n]_+ + \lambda \|\mathbf{w}\|^2$$

where $[\cdot]_+$ clips negative arguments to be zero.

- ▶ The loss-function $E(\mathbf{x}_n, t_n) = [1 - y(\mathbf{x}_n)t_n]_+$ is known as **hinge loss**.
- ▶ Since it is not differentiable, it has been more common in practice to solve the constrained problem (or its dual).

Loss functions



- ▶ We can view all common loss-functions as continuous approximations to the **zero-one-loss** (shown in black), which counts the number of miss-classifications.
- ▶ So, *Boosting*, *Support Vector Machines* and *Logistic Regression* are three very closely related approaches to classification (and they tend to yield comparable classification performance in practice.)

SVM and logistic regression

- ▶ The unconstrained SVM optimization problem is similar to the regularized two-class logistic regression objective (also known as “**log-loss**”):

$$\begin{aligned} E_{\text{logreg}} &:= \sum_{n=1}^N -\log p(t_n|\mathbf{x}_n) + \lambda \|\mathbf{w}\|^2 \\ &= \sum_{n=1}^N \log(1 + \exp(-y(\mathbf{x}_n)t_n)) + \lambda \|\mathbf{w}\|^2 \end{aligned}$$

- ▶ This shows that SVMs and logistic regression are solving very similar problems.
- ▶ In fact, we can rewrite the logistic regression problem as a constrained problem to arrive at a dual, “kernelized” form.
- ▶ A classification approach known as *Boosting*, which we did not cover in this lecture, can be shown to correspond to a third type of loss-function: the “*exponential-loss*”.

Other non-parametric methods

- ▶ There are a variety of non-parametric methods that we did not cover. Examples include:
- ▶ Gaussian processes
- ▶ Support vector regression
- ▶ “Relevance vector machines”
- ▶ Kernel logistic regression/ “import vector machines”